Space Inheritance

DOCUMENT ID	DATE PUBLISHED
US 9323581 B1	2016-04-26

INVENTOR INFORMATION

NAME	CITY	STATE	ZIP CODE	COUNTRY
van Rotterdam; Jeroen	Berkeley	CA	N/A	US
Polman; Mark	Rotterdam	N/A	N/A	NL

ASSIGNEE INFORMATION

NAME	CITY	STATE	ZIP CODE	COUNTRY
EMC Corporation	Hopkinton	MA	N/A	US

TYPE CODE
02

APPLICATION NO	DATE FILED
13/077961	2011-03-31

US CLASS CURRENT:

1/1

CPC CURRENT

TYPE	CPC	DATE
CPCI	G 06 F 3/0644	2013-01-01
CPCI	G 06 F 16/278	2019-01-01
CPCI	G 06 F 3/0631	2013-01-01
CPCI	G 06 F 9/5077	2013-01-01

KWIC Hits

APPLICATION FILING DATE
20110331

Abstract

A method, article of manufacture, and apparatus for managing a cloud computing environment. In some embodiments, this includes partitioning resources to create a space, determining a parent space, inheriting properties of the parent space, and storing the space in a storage device. In some embodiments, a precedence may be designated to resources of the created space, and may override properties of the parent.

Background/Summary

CROSS REFERENCE TO RELATED APPLICATIONS

(1) This application is related to co-pending U.S. patent application Ser. No. 13/077,965 for DISTINGUISHING TENANTS IN A MULTI-TENANT CLOUD ENVIRONMENT and filed concurrently herewith, which is incorporated

herein by reference for all purposes.

FIELD OF THE INVENTION

(2) This invention relates generally to cloud computing systems, and more particularly to systems and methods for partitioning cloud computing systems.

BACKGROUND OF THE INVENTION

(3) Increasingly, consumers of computer resources are utilizing the cloud. Cloud computing generally refers to the provisioning of computer resources through a network at the time a user requests those resources.

(4) Cloud computing provides several benefits. One such benefit is costs savings. Before the cloud, users had to invest in their own hardware and software. This led to a high amount of initial investment in information technology. Further, when a user needed only a small increase in computer resources (e.g. slightly more storage capacity), the user typically could only buy new equipment that provided more resources than required.

(5) Cloud service providers generally have many clients, or users. These clients typically store their data on the cloud service provider. For a variety of reasons, such as privacy, the cloud service provider typically partitions user data and other objects. However, if several users have data in common, some efficiencies of cloud computing are lost when the cloud service provider maintains several copies of the common data. Further, there are overhead costs associated partitioning data. There is a need, therefore, for an improved method, article of manufacture, and apparatus for partitioning cloud computing systems.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

(1) The present invention will be readily understood by the following detailed description in conjunction with the accompanying drawings, wherein like reference numerals designate like structural elements, and in which:

(2) FIG. 1 is a diagram of a space tree in accordance with some embodiments.

(3) FIG. 2 is a diagram of a space tree in accordance with some embodiments.

(4) FIG. 3 is a diagram of a tenant space in accordance with some embodiments.

(5) FIG. 4 is a flowchart of a method to manage a cloud computing environment in accordance with some embodiments.

(6) FIG. 5 is a flowchart of a method to manage a cloud computing environment in accordance with some embodiments.

DETAILED DESCRIPTION

(7) A detailed description of one or more embodiments of the invention is provided below along with accompanying figures that illustrate the principles of the invention. While the invention is described in conjunction with such embodiment(s), it should be understood that the invention is not limited to any one embodiment. On the contrary, the scope of the invention is limited only by the claims and the invention encompasses numerous alternatives, modifications, and equivalents. For the purpose of example, numerous specific details are set forth in the following description in order to provide a thorough understanding of the present invention. These details are provided for the purpose of example, and the present invention may be practiced according to the claims without some or all of these specific details. For the purpose of clarity, technical material that is known in the technical fields related to the invention has not been described in detail so that the present invention is not unnecessarily obscured.

(8) It should be appreciated that the present invention can be implemented in numerous ways, including as a process, an apparatus, a system, a device, a method, or a computer readable medium such as a computer readable storage medium or a computer network wherein computer program instructions are sent over optical or electronic communication links. Applications may take the form of software executing on a general purpose

computer or be hardwired or hard coded in hardware. In this specification, these implementations, or any other form that the invention may take, may be referred to as techniques. In general, the order of the steps of disclosed processes may be altered within the scope of the invention.

(9) An embodiment of the invention will be described with reference to a data storage system in the form of a storage system configured to store files, but it should be understood that the principles of the invention are not limited to this configuration. Rather, they are applicable to any system capable of storing and handling various types of objects, in analog, digital, or other form. Although terms such as document, file, object, etc. may be used by way of example, the principles of the invention are not limited to any particular form of representing and storing data or other information; rather, they are equally applicable to any object capable of representing information.

(10) The enhanced techniques described herein allow for a multi-tenant cloud computing environment to share resources, such as content objects, applications, authorization entities, etc. Sharing of the resources, while offering partitioning, is done in a controlled way with little overhead costs per user.

(11) This is accomplished by using “spaces.” A space is a logical partitioning of resources. A resource may be a hardware resource or a software resource. For example, a resource may be CPU allocation, hard disk storage space, an application, code, or authentication servers, among others.

(12) A space may have child spaces and parent spaces. In addition to partitioning resources, spaces may also organize precedence of resources in some embodiments. For example, in some embodiments, a space lower in a space tree may override or take precedence over a space higher in the space tree.

(13) FIG. 1 illustrates a space tree in accordance with some embodiments. Space Tree **10** includes Spaces **100**, **102**, **104**, **114** and **116**. Space **104** is illustrated to include Type System **106**, Security Policies **108**, and Authentication Server **110**, which are applied to Space **104**. Various other policies may be applied to other spaces, but are not shown in FIG. 1. In Space Tree **10**, Space **104** is a child space of Space **102**, and Space **102** is a parent space of **104**. Space **102** is also a child space of Space **100**. Tenant Space **112** represents the space(s) that are available to a specific user, or tenant.

(14) Depending on the user (e.g. tenant), the number of spaces available may vary greatly. For example, in some embodiments, Space Tree **10** may represent all the spaces available to a corporation. Space **100** may represent the company's general space (may have company wide documents and procedures, biographies of senior management, etc.), Space **102** may represent the legal department's space (may have the legal department's policies and procedures, etc.), and Space **104** may represent a specific attorney's space (may include the personal documents and contact list of the specific attorney). If the specific attorney logged into Space **104** using Space **104**'s authentication policies, in some embodiments, the specific attorney would be granted access to Space **104** and its parent spaces (e.g. Spaces **102** and **100**). The specific attorney would be granted access to Spaces **102** and **100** even if their respective authentication servers were not the same as Space **104**'s. In this case, the specific attorney's tenant space would be Spaces **104**, **102**, and **100**, which is the same as illustrated by FIG. 1 Tenant Space **114**. However, if the specific attorney logged into Space **102** using Space **102**'s authentication server (not shown in FIG. 1), the specific attorney would be granted access to only Space **102** and its parent space (e.g. Space **100**). In this case, the specific attorney's tenant space would be Spaces **102** and **100**. It should be noted that if the attorney logged into Space **102**, the attorney would not be granted access to any sister spaces (Space **114** as illustrated in FIG. 1). Similarly, if someone logged into Space **114**, he or she would not be granted access to Space **104**. Rather, his or her tenant space would include only Spaces **114** and **100**.

(15) Further, in some embodiments, policies may dictate that different tenants see different resources in a space. For example, Space **102**'s policy may dictate that a tenant logged on through Space **104** may view all of Space **102**'s resources, but a tenant logged on through Space **116** may view only a subset of Space **102**'s resources.

(16) Using the enhanced techniques described herein, a cloud service provider may similarly share resources among a large number of users (or tenants) while still partitioning resources among the tenants. For example, a “top” space may include an application accessible to all tenants, and a “low” space may include specific documents created by a specific tenant using the “top” space application. In this way, the cloud service provider does not need a separate copy of the application for each tenant. Overhead is similarly reduced while adding tenants.

(17) In some embodiments, policies may also be used to map spaces to physical partitions. These may be referred to as physical mapping policies. For example, a physical mapping policy may be: Resources starting with “A” are stored in Server X, Directory Y Resources of .doc type are stored in Database Z. Resources by author B are stored in Database C.

As illustrated by the above examples, physical mapping policies identify a set of resources based on some criteria, and at least one physical partition.

(18) In some embodiments, spaces may inherit spaces from their parent spaces. FIG. 2 illustrates how a space may inherit a physical mapping policy from its parent space, and how a space may override its parent's physical mapping policy. As shown in FIG. 2, Space A has the following mapping policies applied: Circle Resources are stored in Server **1** Square Resources are stored in Server **2** Triangle Resources are stored in Database X

Resource **1** is stored in Server **1** since Resource **1** is of a circle type. Similarly, Resource **2** is stored in Server **2** since it is of a square type. Space A is the parent space of Space B and of Space C. Space B inherits its mapping policy from Space A. Accordingly, Resource **4** is stored in Server **1** since Resource **4** is of a circle type, and Resource **6** is stored in Database X since Resource **6** is of a triangle type.

(19) Space C, like Space B, is a child space of Space A, and inherits Space A's mapping policy. However, Space C also has its own policy on how to handle Triangle Resources, and will overrides Space A's policy to the extent the two policies conflict. As illustrated in FIG. 2, Resource **7** is stored in Server **2** since it is a square type resource (as dictated by Space A's policies and inherited by Space C). However, the two policies conflict with regard to how to manage resources of the triangle type. Space A's policy stores triangle resources in Database X, whereas Space C's policy stores triangle resources in Database Y. Since Space C's policy will override Space A's policy, Resources **8** and **9** will be stored in Database Y.

(20) FIG. 4 illustrates a method to manage a cloud computing environment in accordance with some embodiments. In step **400**, resources are partitioned to create a space. In step **402**, a parent space is determined. In step **404**, properties of the parent space are inherited. In step **406**, the space is stored in a storage device. In some embodiments, the resources of the created space may be designed a precedence which allows the created space to override the inherited properties of the parent.

(21) FIG. 3 illustrates the above example through a tenant's view. Suppose Tenant **1** logs onto Space B. Tenant **1**'s tenant space will consist of Space A and Space B. Thus, Tenant **1** will see Resources **1**, **2** and **3** from Space A, and **4**, **5** and **6** from Space B. Further, Tenant **1** will see Resources **1** and **4** are stored in Server **1**, Resources **2** and **5** are stored in Server **2**, and Resource **6** stored in Database X.

(22) Suppose a Tenant **2** logs onto Space C. Tenant **2**'s tenant space will consist of Space A and Space C. Thus, Tenant **2** will see Resources **1**, **2** and **3** from Space a, and **7**, **8** and **9** from Space C. Further, Tenant **2** will see Resources **1** is stored in Server **1**, Resources **2** and **7** are stored in Server **2**, and Resources **8** and **9** are stored in Database Y.

(23) FIG. 5 illustrates a method to manage a cloud computing environment with some embodiments. In step **500**, a tenant space is designated for a tenant. In step **502**, resources are allocated to the tenant in the tenant space based on the tenant. In step **504**, the tenant is associated with an authentication. In step **506**, the association is stored in a storage device. When the tenant is successfully authenticated (e.g. successful username/password login), the tenant is granted access to the resources allocated to the tenant in the tenant space. The tenant cannot see other resources, even the other resources reside in a space in the tenant space.

(24) A cloud service provider may use physical mapping policies to provide different physical resources to different tenants or users. For example, Server **1** may be rated “standard performance,” and may be assigned to spaces accessible by “basic membership” users. Similarly, Server **2** may be rated “high performance,” and may be assigned to spaces accessible only by “premier membership” users.

(25) For the sake of clarity, the processes and methods herein have been illustrated with a specific flow, but it should be understood that other sequences may be possible and that some may be performed in parallel, without departing from the spirit of the invention. Additionally, steps may be subdivided or combined. As disclosed herein, software written in accordance with the present invention may be stored in some form of

computer-readable medium, such as memory or CD-ROM, or transmitted over a network, and executed by a processor.

(26) All references cited herein are intended to be incorporated by reference. Although the present invention has been described above in terms of specific embodiments, it is anticipated that alterations and modifications to this invention will no doubt become apparent to those skilled in the art and may be practiced within the scope and equivalents of the appended claims. More than one computer may be used, such as by using multiple computers in a parallel or load-sharing arrangement or distributing tasks across multiple computers such that, as a whole, they perform the functions of the components identified herein; i.e. they take the place of a single computer. Various functions described above may be performed by a single process or groups of processes, on a single computer or distributed over several computers. Processes may invoke other processes to handle certain tasks. A single storage device may be used, or several may be used to take the place of a single storage device. The disclosed embodiments are illustrative and not restrictive, and the invention is not to be limited to the details given herein. There are many alternative ways of implementing the invention. It is therefore intended that the disclosure and following claims be interpreted as covering all such alterations and modifications as fall within the true spirit and scope of the invention.

Claims

1. A method for managing a cloud computing environment, the method comprising: partitioning resources to create a parent space, and a first child space, the parent space being a logical partition comprising at least a portion of a first server, and a second server, different from the first server; associating a parent physical mapping policy to the parent space, wherein the parent physical mapping policy maps objects of a first type in the parent space to the first server for storage, and maps objects of a second type in the parent space to the second server for storage; associating a first child physical mapping policy to the first child space, wherein the first child physical mapping policy maps objects of the first type in the first child space to a third server for storage; inheriting, at the first child space, properties of the parent space, wherein the inherited properties include the parent physical mapping policy; designating a precedence for the resources of the first child space; overriding, at the first child space, the inherited parent physical mapping policy with the first child physical mapping policy based on the precedence if there is a conflict between the first child physical mapping policy and the inherited parent physical mapping policy; storing the parent space in a storage device; and based on the overriding, storing objects of the first type in the first child space at the third server rather than the first server.

2. The method as recited in claim 1, wherein resources include at least one from the group comprising application data, files, and authentication servers.

3. The method of claim 1, wherein the parent physical mapping policy maps objects of a third type to a database.

4. The method of claim 1 comprising: partitioning the resources to create a second child space; inheriting, at the second child space, the properties of the parent space, wherein the inherited properties at the second child space include the parent physical mapping policy; and storing objects of the first type in the second child space at the first server according to the inherited parent physical mapping policy.

5. The method of claim 1 comprising: authenticating a login by a tenant to the first child space; based on the authenticated login to the first child space, allowing the tenant to access the first child space and the parent space; authenticating a login by the tenant to the parent space; and based on the authenticated login to the parent space, allowing the tenant to access the parent space, and blocking access to the first child space.

6. The method of claim 1 comprising: associating the parent space with a first authentication server; associating the first child space with a second authentication server, different from the first authentication server; upon a tenant attempting to login to the first child space, authenticating the tenant using second authentication server; based on the authentication by the second authentication server, granting the tenant access to the first child space and the parent space; upon a tenant attempting to login to the parent space, authenticating the tenant using the first authentication server; and based on the authentication by the first authentication server, granting the tenant access to the parent space, and blocking access to the first child space.

7. The method of claim 1 comprising: providing a first tenant with access to the first child space; inheriting, at a second child space, properties of the parent space; providing a second tenant, different from the first tenant, with access to the second child space; upon the first tenant logging into the first child space, allowing the first tenant

to access the parent space and blocking the first tenant from accessing the second child space; and upon the second tenant logging into the second child space, allowing the second tenant to access the parent space and blocking the second tenant from accessing the first child space.

8. A system for managing a cloud computing environment, comprising a non-transitory storage device and a processor configured to: partition resources to create a parent space, and a first child space, the parent space being a logical partition comprising at least a portion of a first server, and a second server, different from the first server; associating a parent physical mapping policy to the parent space, wherein the parent physical mapping policy maps objects of a first type in the parent space to the first server for storage, and maps objects of a second type in the parent space to the second server for storage; associating a first child physical mapping policy to the first child space, wherein the first child physical mapping policy maps objects of the first type in the first child space to a third server for storage; inherit, at the first child space, properties of the parent space, wherein the inherited properties include the parent physical mapping policy; designate a precedence for the resources of the first child space; override, at the first child space, the inherited parent physical mapping policy with the first child physical mapping policy based on the precedence if there is a conflict with between the first child physical mapping policy and the feast one inherited parent physical mapping policy; store the parent space in a storage device; and based on the override, store objects of the first type in the first child space at the third server rather than the first server.

9. The system of claim 8, wherein the parent physical mapping policy maps objects of a third type to a database.

10. The system of claim 8 wherein the processor is configured to: partition the resources to create a second child space; inherit, at the second child space, the properties of the parent space, wherein the inherited properties at the second child space include the parent physical mapping policy; and store objects of the first type in the second child space at the first server according to the inherited parent physical mapping policy.

11. The system of claim 8 wherein the processor is configured to: authenticate a login by a tenant to the first child space; based on the authenticated login to the first child space, allow the tenant to access the first child space and the parent space; authenticate a login by the tenant to the parent space; and based on the authenticated login to the parent space, allow the tenant to access the parent space, and block access to the first child space.

12. The system of claim 8 wherein the processor is configured to: associate the parent space with a first authentication server; associate the first child space with a second authentication server, different from the first authentication server; upon a tenant attempting to login to the first child space, authenticate the tenant using second authentication server; based on the authentication by the second authentication server, grant the tenant access to the first child space and the parent space; upon a tenant attempting to login to the parent space, authenticate the tenant using the first authentication server; and based on the authentication by the first authentication server, grant the tenant access to the parent space, and block access to the first child space.

13. The system of claim 8 wherein the processor is configured to: provide a first tenant with access to the first child space; inherit, at a second child space, properties of the parent space; provide a second tenant, different from the first tenant, with access to the second child space; upon the first tenant logging into the first child space, allow the first tenant to access the parent space and blocking the first tenant from accessing the second child space; and upon the second tenant logging into the second child space, allow the second tenant to access the parent space and block the second tenant from accessing the first child space.

14. A computer program product for managing a cloud computing environment, comprising a non-transitory computer readable medium having program instructions embodied therein for: partitioning resources to create a parent space, and a first child space, the parent space being a logical partition comprising at least a portion of a first server, and a second server, different from the first server; associating a parent physical mapping policy to the parent space, wherein the parent physical mapping policy maps objects of a first type in the parent space to the first server for storage, and maps objects of a second type in the parent space to the second server for storage; associating a first child physical mapping policy to the first child space, wherein the first child physical mapping policy maps objects of the first type in the first child space to a third server for storage; inheriting, at the first child space, properties of the parent space, wherein the inherited properties include the parent physical mapping policy; designating a precedence for the resources of the first child space; overriding, at the first child space, the inherited parent physical mapping policy with the first child physical mapping policy based on the precedence if there is a conflict between the first child physical mapping policy and the inherited parent physical

mapping policy; storing the parent space in a storage device; and based on the overriding, storing objects of the first type in the first child space at the third server rather than the first server.

15. The computer program product of claim 14, wherein the parent physical mapping policy maps objects of a third type to a database.

16. The computer program product of claim 14 wherein the non-transitory computer readable medium comprises program instructions embodied therein for: partitioning the resources to create a second child space; inheriting, at the second child space, the properties of the parent space, wherein the inherited properties at the second child space include the parent physical mapping policy; and storing objects of the first type in the second child space at the first server according to the inherited parent physical mapping policy.

17. The computer program product of claim 14 wherein the non-transitory computer readable medium comprises program instructions embodied therein for: authenticating a login by a tenant to the first child space; based on the authenticated login to the first child space, allowing the tenant to access the first child space and the parent space; authenticating a login by the tenant to the parent space; and based on the authenticated login to the parent space, allowing the tenant to access the parent space, and blocking access to the first child space.

18. The computer program product of claim 14 wherein the non-transitory computer readable medium comprises program instructions embodied therein for: associating the parent space with a first authentication server; associating the first child space with a second authentication server, different from the first authentication server; upon a tenant attempting to login to the first child space, authenticating the tenant using second authentication server; based on the authentication by the second authentication server, granting the tenant access to the first child space and the parent space; upon a tenant attempting to login to the parent space, authenticating the tenant using the first authentication server; and based on the authentication by the first authentication server, granting the tenant access to the parent space, and blocking access to the first child space.

19. The computer program product of claim 14 wherein the non-transitory computer readable medium comprises program instructions embodied therein for: providing a first tenant with access to the first child space; inheriting, at a second child space, properties of the parent space; providing a second tenant, different from the first tenant, with access to the second child space; upon the first tenant logging into the first child space, allowing the first tenant to access the parent space and blocking the first tenant from accessing the second child space; and upon the second tenant logging into the second child space, allowing the second tenant to access the parent space and blocking the second tenant from accessing the first child space.